# Practical 1

**Aim/Objective:**

1. Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.

2. Write a program to implement Missionaries and Cannibals problem.Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on Board.

**Theory:**

The 2-water jug problem and Missionaries and Cannibals problem are classic examples of State space search and AI the demonstrates the constraints that affect the problem solving. In 2-water jug problem, we have 2 jugs in which the subject has to create an algorithm for moving from one location (the empty two water jugs) to another (filled).In problem 2 i.e Missionaries and Cannibals problem it involves a group of missionaries and cannibals must cross a river in a boat where the number of passengers allowed on the boat is not known.The key constraint is that at no point on either side of the river should cannibals outnumber missionaries,In the starting configuration, all three pairs of missionaries and cannibals are found at the starting point (the left bank) and the boat is also on the left bank.The goal state is achieved when all missionaries and cannibals are safely transferred to the right bank.The methods for producing potential moves and determining valid moves depend upon size of the boat's interior; although, it will always allow for at most two passengers to cross in one trip.After each move, the resulting state is checked against the problem constraints to ensure it is safe and valid.Any state failing to satisfy all conditions will not be retained in memory.Thus, the implementation focuses on defining states, generating valid transitions, enforcing constraints, and applying a search strategy to reach the goal state efficiently.

**Algorithm/Methodology:**

**1. ALGORITHM: TWO WATER JUG PROBLEM**

i) Start with the initial state (0, 0), where both jugs are empty.

ii) The goal state is one of the jug having the desired water quantity.

iii) From the current state, generate all possible next states by applying the allowed operations:
Fill Jug A completely.

Fill Jug B completely.
Empty Jug A.
Empty Jug B.
Transfer water from Jug A
Transfer water from Jug B to Jug A.

iv) for each newly generated state:
Check whether it satisfies the goal condition. If yes, stop the process and report the solution.

v) Repeat this process until the goal state is reached or all possible states have been explored.

**2. ALGORITHM: MISSIONARIES AND CANNIBALS PROBLEM:**

i) Represent each state as (M, C, B), where M is the number of missionaries and, C is the number of cannibals on the left bank, and B indicates the position of the boat.

ii) Initial state (3, 3, Left), where all missionaries, cannibals, and the boat are on the left bank.

iii) Define the goal state as (0, 0, Right), means everyone has crossed to the right bank.

iv) Valid moves of the boat, allowing one or two people to cross at a time. The possible combinations include:
o One missionary or two missionaries
o One cannibal or two cannibals
o One missionary and one cannibal

v) After each move, update the state based on the direction of the boat and check the constraints:
o On the either side, the number of missionaries should not be less than number of cannibals.

vi) If the new state is valid and has not been visited , add it to the search.

vii) Continue this process until the goal state is reached or no further valid states can be generated.

**Dataset Description:**

1.Source: user given

2.Platform: Google Collab

3.Language: Python

**Implementation:**

**CODE: 2-WATER JUG PROBLEM:**

```
j1 = 0
j2 = 0
capx = 3
capy = 5


print("step 1 = fill x")
print("step 2 = fill y")
print("step 3 = empty x")
print("step 4 = empty y")
print("step 5 = pour x into y")
print("step 6 = pour y into x")
print("step 7 = exit")


while True:
   step = int(input("Enter the step no: "))


   if step == 1:
       j1 = capx
       print("Filling x")
       print(f"jar1={j1} and jar2={j2}")

   elif step == 2:
       j2 = capy
       print("Filling y")
       print(f"jar1={j1} and jar2={j2}")

   elif step == 3:
       j1 = 0
       print("Empty x")
       print(f"jar1={j1} and jar2={j2}")

   elif step == 4:
```

```
        j2 = 0
        print("Empty y")
        print(f"jar1={j1} and jar2={j2}")

    elif step == 5:
        print("Pouring x into y")
        transfer = min(j1, capy - j2)    # how much can be poured
        j1 -= transfer
        j2 += transfer
        print(f"jar1={j1} and jar2={j2}")

    elif step == 6:
        print("Pouring y into x")
        transfer = min(j2, capx - j1)    # how much can be poured
        j2 -= transfer
        j1 += transfer
        print(f"jar1={j1} and jar2={j2}")

    elif step == 7:
        print("Exiting program")
        break

    else:
        print("Invalid step")
```

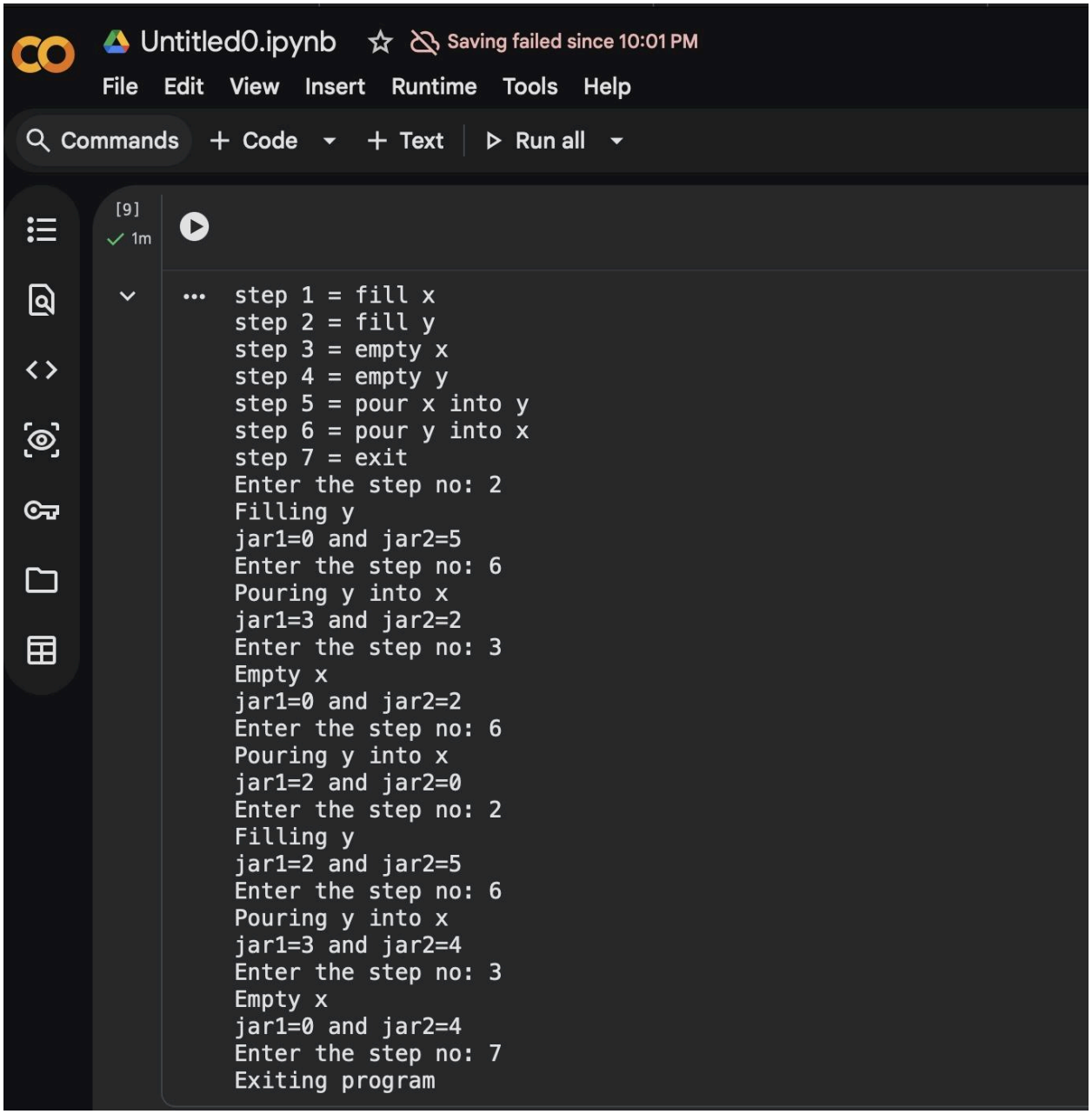
**CODE : MISSIONARIES AND CANNIBALS PROBLEM**

```
from collections import deque


def is_valid(m, c):
    return (m == 0 or m >= c) and (3 - m == 0 or 3 - m >= 3 - c)


def missionaries_cannibals():
    start = (3, 3, 0)
    goal = (0, 0, 1)
```

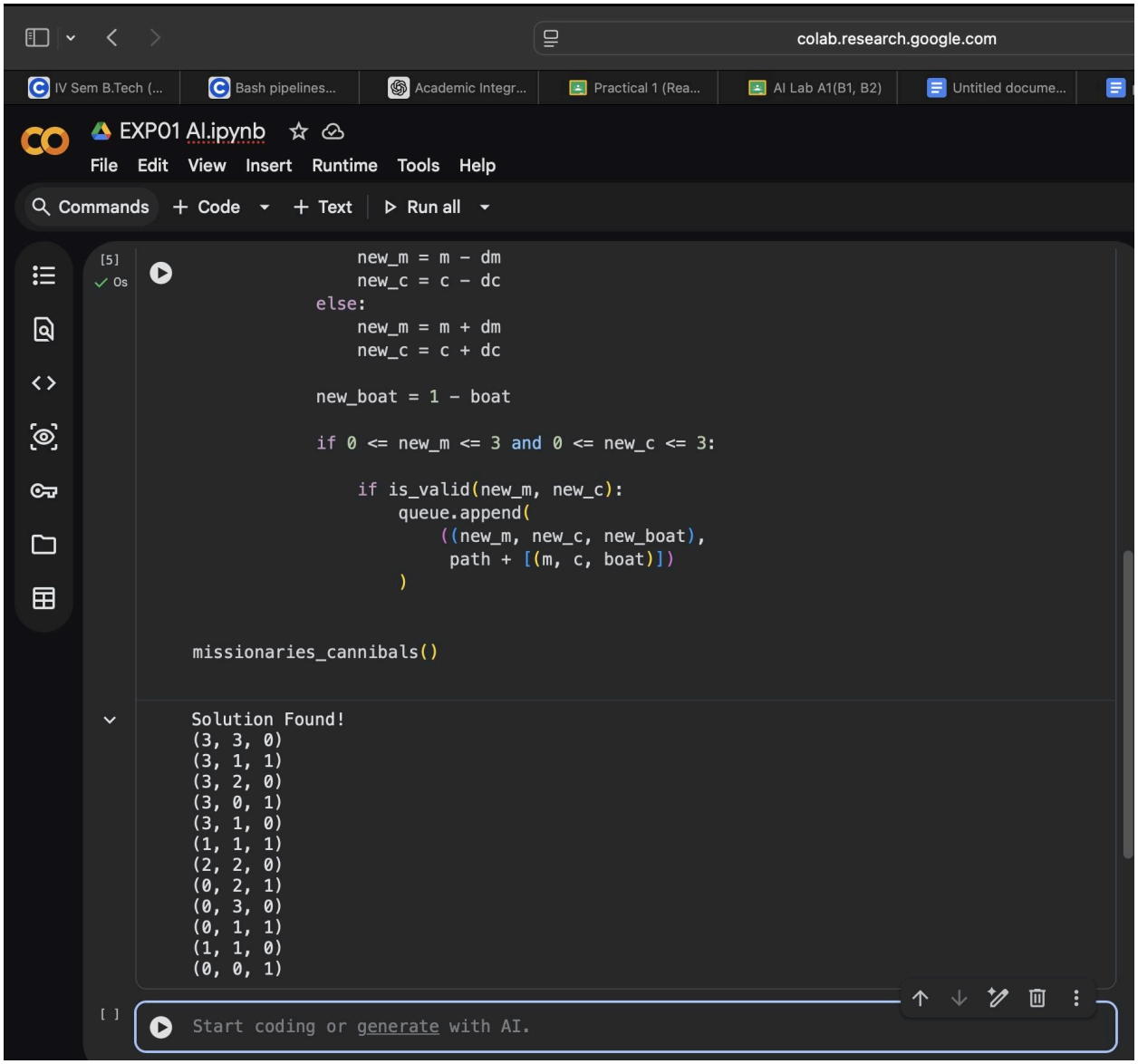
```
    queue = deque([(start, [])])
    visited = set()

    while queue:

        (m, c, boat), path = queue.popleft()

        if (m, c, boat) == goal:
            print("Solution Found!")
            for step in path + [(m, c, boat)]:
                print(step)
            return

        if (m, c, boat) in visited:
            continue

        visited.add((m, c, boat))

        for dm, dc in [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]:

            if boat == 0:
                new_m = m - dm
                new_c = c - dc
            else:
                new_m = m + dm
                new_c = c + dc

            new_boat = 1 - boat

            if 0 <= new_m <= 3 and 0 <= new_c <= 3:

                if is_valid(new_m, new_c):
                    queue.append(
                        ((new_m, new_c, new_boat),
                         path + [(m, c, boat)])
                    )


missionaries_cannibals()
```

Output/Result:

1.TWO WATER JUG PROBLEM




**2.MISSIONARIES AND CANNIBALS PROBLEM**




**Result Analysis:**

The implementation of the Two Water Jug problem and the Missionaries and Cannibals problem were defining their valid operations.In the Two Water Jug Problem, the algorithm produced every configuration of jugs by performing operations (filling, pouring out, and transferring) between the two jugs in every possible way.In the Missionaries and Cannibals problem, the algorithm successfully generated a safe sequence of river crossings.

**Conclusion**:

Both 2-jug problem and missionaries and cannibals problem shows significance of BFS (Breadth-First Search) in artificial intelligence.